

# Unveiling the Hidden Echoes: Zero-Dependency Forensic Reconstruction of 'Deleted' LLM Chats from Browser & Desktop LevelDB Instances

Dr. Sapna Vikram Mewundi<sup>\*</sup>  
*PES University, Bengaluru, India*  
Email: sapnavm@pes.edu

## Abstract

As Large Language Model (LLM) portals become standard corporate utilities, proprietary source code, credentials, and sensitive configurations are routinely processed by users. When a user clicks 'Delete Chat' inside ChatGPT, Claude, or Gemini interfaces, they expect their local trace data to be permanently erased. However, because Chromium-based browsers (Chrome, Edge) and Electron desktop clients store this telemetry inside IndexedDB databases backed by Google's LevelDB engine, these deleted records persist on disk in write-ahead logs (.log) and uncompact Sorted String Tables (.sst/ldb). This paper details the reverse engineering of client-side LLM storage schemas and the V8 deserialization format. We present the mechanics of rebuilding conversation trees directly from raw binary fragments. Finally, we introduce a zero-dependency, pure-Python carving methodology to bypass active database locks and dynamically reconstruct deleted chat histories across all user profiles, providing incident responders with a secure, cryptographically validated chain of custody.

## I. INTRODUCTION

The security industry has focused heavily on Generative AI security at the boundary layer: prompt injections, data guardrails, and cloud storage compliance. Virtually unmapped, however, is the client-side forensic footprint left behind by web interfaces and native wrapper applications on employee workstations.

When a user deletes a chat thread, the cloud-side storage is updated. Locally on the endpoint, however, the deletion is simply written as a LevelDB tombstone or index update. Because LevelDB is an append-only log-structured merge-tree (LSM tree), the actual serialized data blocks containing the prompts and responses remain intact in write-ahead log records or orphaned data blocks until compaction occurs. This creates a critical forensic window: an attacker or examiner with local access can recover sensitive, supposedly 'deleted' data.

## II. LOW-LEVEL DATABASE ARCHITECTURE: INDEXEDDB & LEVELDB

Chromium browsers implement IndexedDB using LevelDB as the underlying storage engine. On Windows, these files reside in user AppData folders:

```
%LOCALAPPDATA%\Google\Chrome\User  
Data\<ProfileName>\IndexedDB\https_chatgpt.com_0.indexeddb.leveldb\
```

Traditional forensics tools rely on loading LevelDB using compiled C++ drivers (like plyvel). During a live incident response triage, installing compiler tools on production machines is prohibited. Incident responders require a zero-dependency byte carver that directly dissects raw files.

---

<sup>\*</sup> Dr. Sapna Vikram Mewundi is with the Department of Computer Science & Engineering, PES University, Bengaluru, India. Email: sapnavm@pes.edu

### III. REVERSING V8 SERIALIZATION: THE BINARY STRUCTURE

Chrome and Edge store IndexedDB records serialized in V8's internal binary serialization format (ValueSerializer).

#### A. Varint Decoding

V8 uses Google Protocol Buffer style varints (variable-length integers) to encode lengths and indexes. Every byte's highest-order bit (bit 7) is a flag: 1 means another byte follows, and 0 indicates the end of the varint. The lower 7 bits of each byte are concatenated to form the integer value.

#### B. V8 String Tags and UTF-16LE Encoding

Strings in the serialized format are prefixed with data type tags: OneByteString (0x22) indicates ASCII string, and TwoByteString (0x63) indicates UTF-16LE string. Crucially, the varint length of TwoByteString represents the number of bytes, not characters. The string is decoded by reading exactly length bytes and translating them using UTF-16LE.

### IV. MESSAGE REASSEMBLY & ROLE RECONSTRUCTION

To map raw strings into chronological conversations, we reverse-engineered the serialization structure of the messages array.

#### A. Smi-Shifted Index Keys

V8 serializes array element properties using small integers (Smi) as keys. V8 encodes Smis by shifting the integer left by 1 bit (encoded = actual << 1). The actual message index indicates the role: Odd actual indices represent User Prompts, and Even actual indices represent Assistant Responses. The role mapping is mathematically defined as:

```
role = 'user' if (index / 2) mod 2 = 1 else 'assistant'
```

#### B. Nesting Depth Tracking

V8 serialized message objects contain nested objects. Because object ends are marked by 0x7b, a simple parser would break out of the message parsing loop prematurely. A forensic-grade parser must track nesting depth, incrementing on object (0x6f) or array (0x61) tags, and decrementing on object end (0x7b).

### V. THE VERITY FORENSIC FRAMEWORK

To operationalize these findings, we developed Verity, a zero-dependency, pure-Python carving tool. It bypasses Windows exclusive write locks by accessing write-ahead logs in read-only binary mode, cloning files dynamically, and reassembling conversation threads across all user profiles. To ensure evidence integrity, Verity seals the output using HMAC-SHA256 signatures, validated in-browser via the Web Cryptography API.

## **VI. CONCLUSION**

This paper demonstrates that client-side ephemeral AI sessions are a myth. By parsing raw V8 serialized objects from LevelDB logs, investigators can fully recover deleted records. The Verity framework provides incident responders with a secure, lightweight tool to audit endpoints and protect corporate data assets.